

어려운 문제를 **AI**로 해결하는 전문가 **AI**로 더 나아진 세상을!

20260701(수) 19:00-20:00 차성재

(패션 플랫폼 M사 **Agentic AI PM**, 시립대/아주대 **AI** 부문 겸임교수)

AI Campus Prism 사전 특강

기획·디자인·개발·데이터·인프라·보안이 AI Native로 연결되는 60분

AI Native

Problem

Product

Platform

People

Agentic AI PM Lecture

60분 운영안

45~50분은 강의, 10~15분은 사전 질문과 실시간 Q&A로 구성합니다.

00-05	오프닝	AI가 바꾸는 문제 해결의 관점
05-15	어려운 문제	모호함, 맥락, 스케일, 리스크를 다루는 법
15-25	Agentic AI	모델, 도구, 메모리, 평가, 가드레일
25-38	역할 변화	PM, UX/UI, Engineering, AI-side가 함께 일하는 방식
38-50	커리큘럼 연결	AI Native 기술 키워드를 현업 문제로 번역
50-60	Q&A	사전 질문 4개 + 실시간 질문

오늘의 청중: 이미 질문이 좋습니다

사전 질문은 전공, 직무, 현업 사례, 취업 역량으로 모입니다.

QUESTION 01

UX 전공자는 **AX** 시대에
무엇을 준비해야 하나

기업이 보는 역량은 화면 제작보다 문제 정의,
실험, 데이터, 프로토타이핑으로 확장됩니다.

QUESTION 02

빌더와 디자인+엔지니어링은
주류가 될까

모든 사람이 풀스택이 되는 것은 아니지만,
경계에서 빠르게 검증하는 역할은 커집니다.

QUESTION 03/04

직군 변화와 실제 활용 사례

추천, 검색, 리뷰, 운영, 보안, 내부 지식 등
업무 자체가 AI와 다시 설계됩니다.

어려운 문제는 기술 문제가 아닙니다

정답이 없고, 이해관계자가 많고, 실패 비용이 있으며, 계속 변하는 문제가 어렵습니다.

AI가 잘 도와주는 문제의 조건

AI는 답을 주는 기계가 아니라, 복잡한 탐색을 빠르게 반복하게 해주는 협업 인터페이스입니다.

AMBIGUITY

요구가 모호하다

사용자 말, 문서, 로그, 리뷰에서 의도와 제약을 구조화해야 하는 문제

SCALE

정보가 너무 많다

상품, 리뷰, CS, 정책, 문서처럼 사람이 다 읽기 어려운 문제

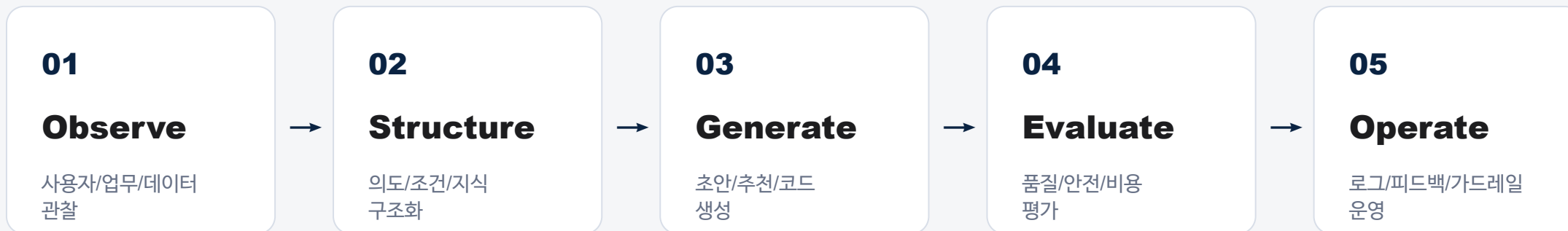
FEEDBACK

정답이 계속 바뀐다

추천 품질, 검색 만족도, 응답 안전성처럼 운영 중 개선해야 하는 문제

AI 시대의 문제 해결 루프

좋은 AI 제품은 한 번에 완성되지 않고, 관찰-구조화-생성-검증-운영의 루프를 빨리 돌립니다.



AI는 ‘기능’ 이 아니라 문제 해결 스택입니다

프롬프트 하나가 아니라 데이터, 도구, 평가, 운영이 결합될 때 현업 가치가 생깁니다.

Experience

사용자가 만나는 화면·대화 흐름

Agent

목표를 이해하고 도구를 선택하는 실행 흐름

Knowledge

문서, 상품, 로그, 정책, 리뷰를 검색 가능한 구조로

Evaluation

정확성, 유용성, 안전성, 비용을 측정

Platform

배포, 관측성, 보안, 권한, 장애 대응

핵심: AI Native는 도구 사용 능력이 아니라, 이 스택을 제품/업무에 맞게 조합하는 능력입니다.

Agentic AI를 한 문장으로

사용자의 목표를 받아, 필요한 지식과 도구를 선택하고, 실행 결과를 다시 판단하는 소프트웨어 패턴입니다.

GOAL

목표를 이해한다

단순 질의응답을 넘어 의도, 제약, 성공 기준을 분해합니다.

TOOLS

도구를 호출한다

검색, DB, 코드 실행, 업무 시스템, API를 필요에 따라 연결합니다.

CONTROL

결과를 검증한다

에러, 권한, 안전성, 비용, 품질을 확인하며 루프를 멈춥니다.

에이전트의 기본 루프

Planner, Tools, Memory, Evaluator, Guardrail이 모이면 실무형 AI 흐름이 됩니다.



이 루프가 제품 안으로 들어오면 PM은 정책을, 디자이너는 인터랙션을, 엔지니어는 실행/검증을 설계하게 됩니다.

최신 AI 기술을 현업 언어로 번역하면

기술명보다 중요한 것은 어떤 책임을 맡기는지입니다.

Tool Use / Function Calling

AI가 외부 API와 업무 시스템을 호출하게 만드는 기술

MCP

도구와 데이터 소스를 AI 클라이언트에 연결하는 표준 인터페이스

RAG / Vector DB

사내 문서, 상품정보, 리뷰를 검색 가능한 지식으로 바꾸는 구조

Agent Workflow

여러 단계의 판단과 실행을 상태 기반으로 운영하는 패턴

Evals / Observability

정확성, 지연 시간, 비용, 실패 케이스를 측정하는 체계

Guardrails / Red Teaming

프롬프트 공격, 데이터 유출, 위험 응답을 막는 안전 장치

직무는 사라지기보다 다시 배치됩니다

AI를 쓰는 사람이 아니라, AI가 들어온 업무 흐름을 설계하고 검증하는 사람이 필요해집니다.

PM: 요구사항이 아니라 ‘판단 시스템’ 을 설계합니다

AI PM은 화면 기능을 나열하는 사람이 아니라, 모델이 무엇을 근거로 어떤 행동을 해야 하는지 정합니다.

INPUT

의도와 제약 구조화

사용자 문장을 목적, 예산, 상황, 선호, 회피 조건으로 바꿉니다.

POLICY

추천/응답 기준 정의

좋은 답변, 위험한 답변, 모르는 답변의 기준을 PRD에 씁니다.

METRIC

선행 지표 설계

정답률만 보지 않고 신뢰도, 재질문율, 선택 확신도, 저장률을 봅니다.

UX/UI Design: 화면보다 ‘불확실성의 상태’ 를 디자인합니다

AI 제품은 늘 로딩, 확인, 재질문, 오류, 보류, 설명 가능성의 상태를 가집니다.

STATE

AI 응답 상태 설계

생성 중, 근거 확인, 실패, 안전 차단, 사람 확인 요청을 명확히 보여줍니다.

TRUST

근거와 선택 확신

왜 추천했는지, 어떤 조건에는 맞지 않는지, 사용자가 판단할 단서를 줍니다.

SYSTEM

토큰과 컴포넌트

디자인 시스템이 프롬프트와 코드 생성의 공통 언어가 됩니다.

Engineering: 코드를 빨리 쓰는 것보다 빨리 검증하는 사람이 강합니다

BE/FE/Client/App 개발은 AI 생성 결과를 읽고, 경계 조건을 테스트하고, 운영 가능한 구조로 바꾸는 쪽으로 이동합니다.

BE

API·데이터·권한

Function Calling, 인증, 로그, 예외, 데이터 계약을 안전하게 설계합니다.

FE/CLIENT

AI 인터랙션

스트리밍, optimistic UI, 오류 복구, 접근성, 상태 관리를 구현합니다.

QUALITY

테스트 자동화

E2E, 평가셋, 배포 게이트로 AI가 만든 코드를 제품 수준으로 끌어올립니다.

AI-side / Platform: AI가 계속 일하게 만드는 사람들

모델 호출은 시작일 뿐입니다. 실제 서비스는 데이터, 서빙, 관측성, 보안, 비용 관리로 지속됩니다.

DATA

RAG와 데이터 품질

문서 최신성, 메타데이터, 권한, 피드백
로그를 운영합니다.

INFRA

LLM 서빙과 관측성

GPU, vLLM, Kubernetes,
Prometheus/Grafana, SLO를 다룹니다.

SECURITY

가드레일과 레드팀

프롬프트 공격, 도구 오남용, 데이터 유출을
지속적으로 검증합니다.

현업 적용은 작은 자동화에서 시작해 제품 구조로 갑니다

패션 플랫폼, 커머스, 교육, 금융, 공공 어디든 출발점은 ‘고객의 판단 비용을 줄이는 일’입니다.

패션 플랫폼 현업에서 만나는 AI 과제

공개 가능한 수준으로 일반화하면, AI는 탐색·판단·운영·품질의 비용을 줄이는 데 쓰입니다.

검색/추천	자연어 의도, 취향, 상황, 가격, 재고를 함께 해석
상품정보 구조화	상품명, 속성, 리뷰, FAQ를 AI가 이해 가능한 형태로 개선
리뷰/콘텐츠 요약	장점, 단점, 사이즈, 소재, 착용감, 주의 조건을 추출
운영 자동화	CS, 정책, 내부 지식, 이슈 분류, 리포트 초안 작성
품질/안전	잘못된 추천, 민감정보, 브랜드 리스크, hallucination 관리

실무형 프로젝트 구조에서 배울 점

프로젝트는 기술 과시가 아니라 문제, 데이터, 기능, 지표, 산출물이 이어지는 구조여야 합니다.

- 주제: AI 쇼핑 발견 도우미 설계 및 구현
- 핵심 기능: 구매 의도 기반 Top 3~5 추천, AI 추천 준비도 진단, 리뷰 기반 구매 불안 요약
- 산출물: PRD, 배포 URL, 랜딩 페이지, 사용자 인터뷰, 인사이트 리포트, 최종 발표
- 지표: 추천 클릭률, 저장률, 선택 확신도, 구매 의향 점수, 모의 장바구니 담기율

이 구조를 **AI Campus Prism** 프로젝트에도 그대로 적용하면 포트폴리오가 ‘작품’ 이 아니라 ‘검증된 문제 해결 기록’ 이 됩니다.

PRD에서 MVP까지: AI 빌더의 작업 흐름

아이디어를 서비스로 바꾸는 사람은 생성형 AI를 ‘초안 공장’이 아니라 ‘검증 루프 가속기’로 씁니다.



핵심은 ‘만들었다’보다 ‘왜 이 기능이 맞는지, 어떻게 측정했는지, 어디까지 안전한지’를 설명하는 능력입니다.

AI 프로젝트의 좋은 지표

AI 제품은 정답률만으로 판단하지 않습니다. 사용자의 판단 비용과 운영 리스크가 함께 줄어야 합니다.

USER VALUE

판단 비용 감소

탐색 시간, 추가 검색 필요성, 선택 확신도,
저장률, 재방문 의향

MODEL QUALITY

응답 품질

근거 일치, 관련성, 재질문율, 실패 케이스,
검색 품질 평가

OPERATION

운영 가능성

지연 시간, 비용, 오류율, SLO, 보안 차단율,
가드레일 위반율

커리큘럼 키워드를 현업 언어로 바꿔보기

AI Campus Prism의 네 과정은 서로 다른 직무가 아니라 하나의 AI 서비스 운영 체인을 나눠 배웁니다.

4개 과정은 하나의 **AI** 서비스 조직입니다

각 과정은 다른 출발점을 갖지만, 최종 프로젝트에서는 서로의 산출물이 연결됩니다.

AI 인프라 운영 및 설계 엔지니어

Kubernetes, GPU/LLM 서빙, Observability, GitOps

AI 네이티브 보안 엔지니어

Threat Modeling, Prompt Attack, Red Teaming, Guardrail

AX 서비스 풀스택 빌더

PRD, FastAPI, React, RAG, Function Calling, CI/CD

AI 네이티브 데이터 플랫폼 엔지니어

Lakehouse, Airflow, Kafka, Data Quality, Vector DB, DataOps

AX 서비스 풀스택 빌더 과정의 현업 번역

아이디어를 AI 서비스 MVP로 완성하는 사람에게 필요한 언어입니다.

PLAN

AX 프로젝트 기획

사용자 리서치, 페르소나, 여정 맵,
MoSCoW, 사용성 테스트

BUILD

AI-Augmented 개발

FastAPI, React, ORM, API 스펙, 인증, AI
코드 리뷰

AI

LLM 서비스 엔지니어링

LangChain, RAG, Vector DB, 검색 품질
평가, Function Calling, 스트리밍

데이터 모델·플랫폼 과정의 현업 번역

AI가 똑똑해 보이는 순간 뒤에는, 대개 데이터 구조와 품질을 만든 사람이 있습니다.

MODEL

시맨틱 모델링

SQL, 스타 스키마, 지표 거버넌스, dbt,
데이터 리니지

PIPELINE

Lakehouse·Streaming

Raw/Staging/Mart, Airflow DAG,
Kafka/Spark, Backfill, Retry

RAG DATA

문서 데이터 플랫폼

Chunking, Embedding, Metadata,
Hybrid Search, 문서 최신성, 피드백 로그

인프라 과정의 현업 번역

AI 서비스는 모델보다 먼저 시스템입니다. 느리고 불안정하면 아무도 쓰지 않습니다.

SERVE

GPU/LLM 서빙

GPU, VRAM, CUDA, vLLM, Triton, 배치 처리, 지연 시간, 처리량

OBSERVE

관측성과 SRE

Log, Metric, Trace, Prometheus, Grafana, SLI/SLO/SLA

RELEASE

GitOps 운영

GitHub Actions, ArgoCD, Blue/Green, Canary, Rollback, Runbook

보안 과정의 현업 번역

AI를 더 많이 쓰려면, 더 정교하게 막고 감시하고 설명할 수 있어야 합니다.

THREAT

AI 위협 모델링

LLM/RAG/Agent 구조, 공격 표면, 데이터 흐름, 보안 요구사항

ATTACK

레드팀과 취약점 검증

Prompt Attack, Jailbreak, Tool Abuse, Data Exfiltration, 평가셋

DEFENSE

가드레일 운영

입력/출력 정책, 권한 필터링, 위험 등급, 보안 대시보드, 알림

AI Native 팀의 협업 모델

좋은 AI 서비스는 한 직군의 천재성이 아니라, 역할 간 인터페이스가 명확할 때 만들어집니다.

PM

문제·정책·지표

UX/UI

상태·신뢰·흐름

FE/BE

API·상태·검증

DATA

지식·품질·피드백

INFRA

서빙·관측·배포

SECURITY

권한·공격·가드레일

인터페이스 문서: **PRD, API Spec, Data Contract, Eval Set, Runbook, Guardrail Policy**

포트폴리오는 결과물이 아니라 의사결정 기록입니다

채용자는 ‘무엇을 만들었나’보다 ‘왜 그렇게 만들었고 어떻게 검증했나’를 봅니다.

01

문제 정의

누구의 어떤 판단 비용을 줄였는지 한
문장으로 설명

02

AI 설계

프롬프트, 도구, RAG, 정책, 실패 처리를
구조로 설명

03

검증과 학습

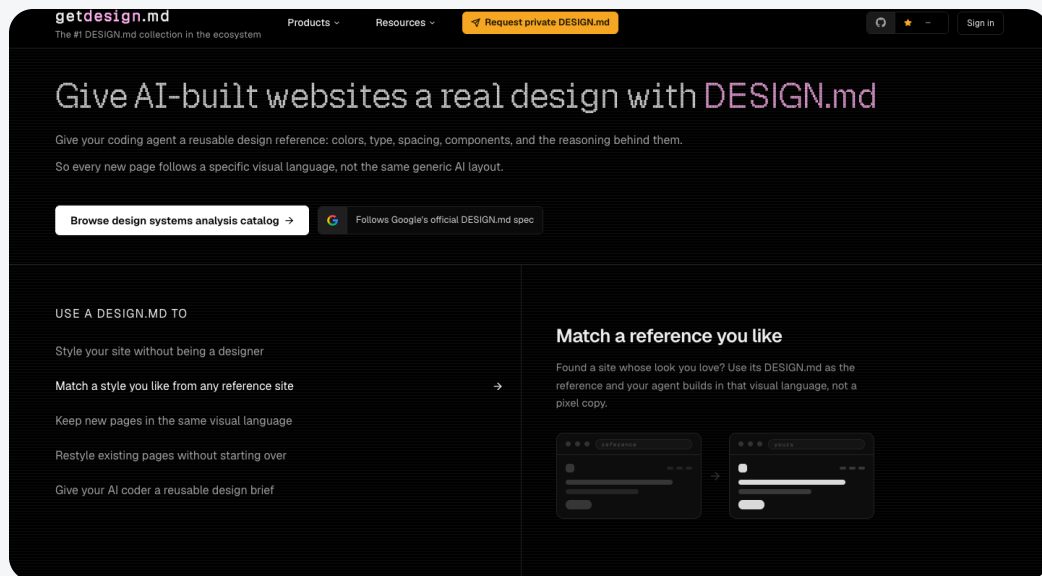
사용자 테스트, 평가셋, 로그, 지표 변화,
다음 개선안

참고 기술과 디자인 레퍼런스

아래 레퍼런스들은 ‘무엇을 베꼈는가’가 아니라, 어떤 기준으로 설계 언어와 기술 언어를 정리했는가를 보여줍니다.

Design System: getdesign.md + Apple tokens

이번 자료는 로컬 DESIGN-apple.md의 토큰을 발표용 디자인 시스템으로 재구성했습니다.



<https://getdesign.md/>

- Deep Navy #0A2342
- white / parchment / near-black rhythm
- 큰 헤드라인, 낮은 밀도, 깊은 여백
- full-bleed section 중심
- tokens + components saved in designs/

AI Campus Prism 공식 페이지

지원자에게는 과정의 분위기와 신뢰도를 확인할 수 있는 가장 직접적인 참고자료입니다.



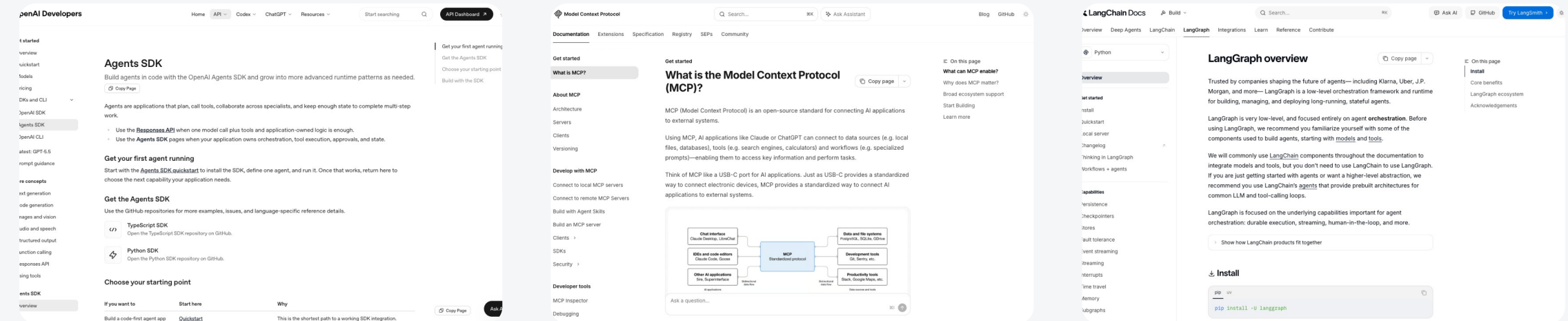
<https://aicampus.goorm.io/>

강의 중 연결 포인트

- AI 실무형 부트캠프
- 지원자·예비 지원자 대상
- 커리어 전환의 불안보다 학습 루프 강조

Agent Stack Reference

에이전트 제품은 모델, 도구 연결, 워크플로우 상태 관리가 함께 필요합니다.



OpenAI Agents SDK

Model Context Protocol

LangGraph

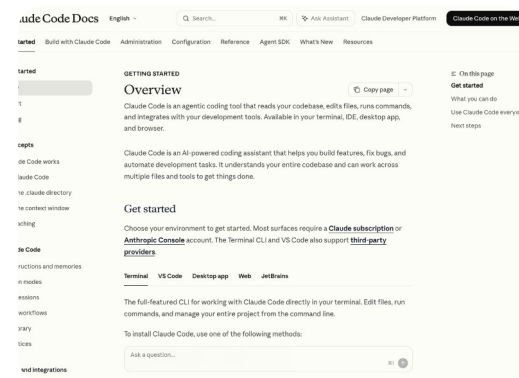
현업 해석: AI가 외부 도구를 호출하고, 상태를 유지하며, 실패와 권한을 통제하는 구조를 설계한다.

Builder Stack Reference

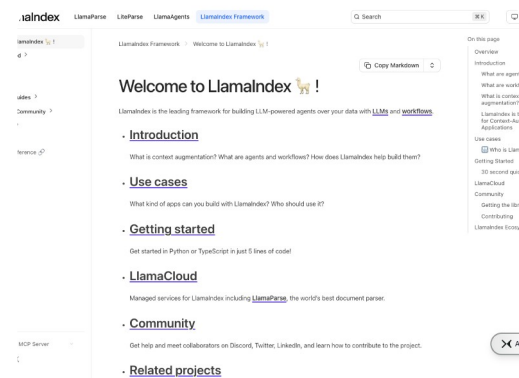
기획-디자인-구현-배포를 빠르게 오가려면, 도구를 연결하는 감각이 필요합니다.



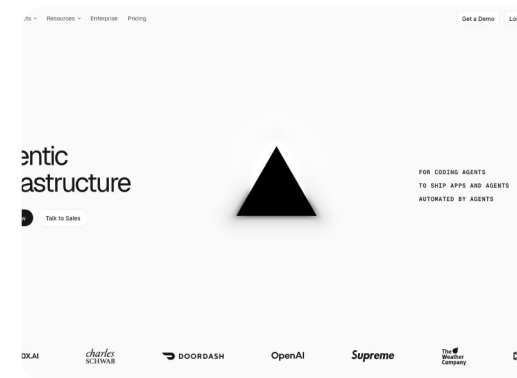
Figma AI
아이디어 → 프로토타입



Claude Code
코드 협업



LlamaIndex
RAG·데이터 연결

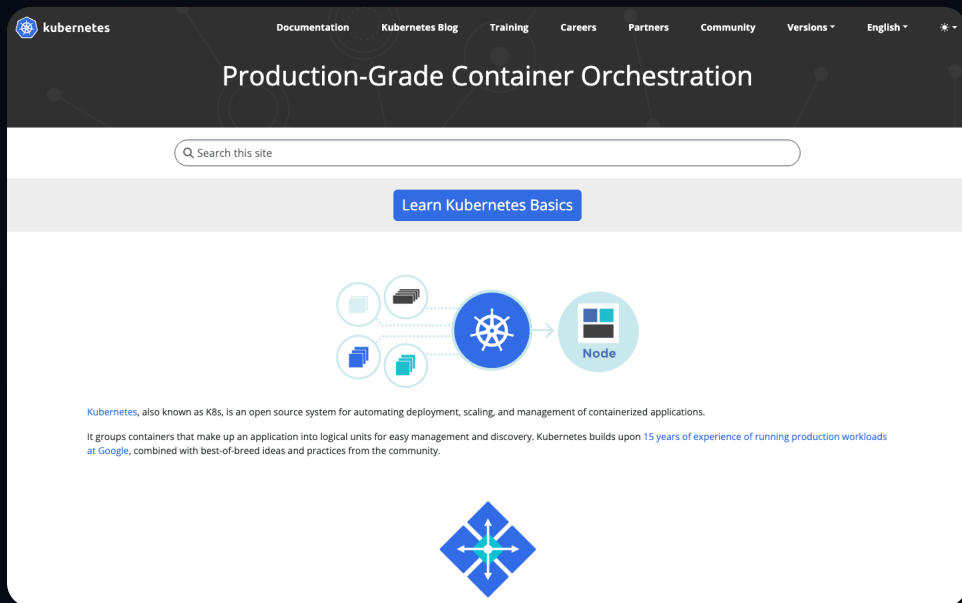


Vercel
배포·프론트엔드

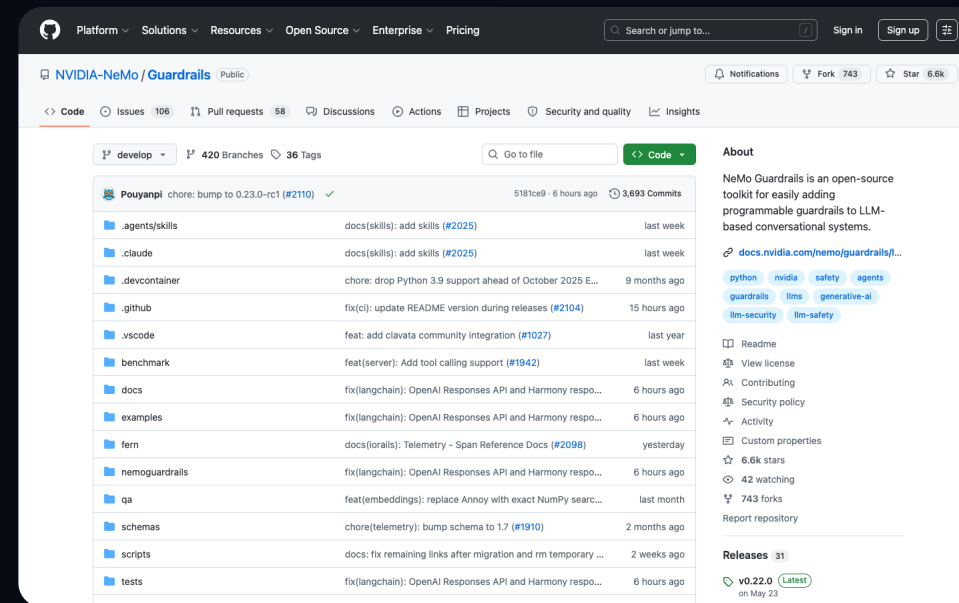
현업 해석: 디자인, 코드, 지식 베이스, 배포가 분리된 단계가 아니라 빠른 검증 루프로 연결됩니다.

Operate & Secure Reference

AI 서비스는 출시 이후가 진짜 시작입니다. 운영과 보안은 AI Native의 기본기입니다.



Kubernetes: AI 서비스 실행과 확장



NeMo Guardrails: 안전한 LLM 운영

5개월을 이렇게 쓰면 강해집니다

커리큘럼을 완주하는 동안, 매주 하나씩 포트폴리오의 증거를 쌓으세요.

1. 매주 하나의 문제를 한 문장으로 재정의한다.
2. AI가 판단할 기준과 실패 기준을 문서화한다.
3. 생성 결과를 그대로 쓰지 말고 평가셋으로 검증한다.
4. 로그와 사용자 피드백으로 다음 개선안을 만든다.
5. 최종 발표는 기능 시연보다 의사결정의 흐름을 보여준다.

사전 질문에 답변하겠습니다

정답처럼 단정하기보다, 앞으로 기업이 무엇을 더 자주 확인하게 될지 기준으로 답합니다.

Q1. UX 디자인 전공자는 무엇을 준비해야 하나

기업은 ‘툴을 잘 쓰는 사람’보다 ‘불확실한 문제를 검증 가능한 형태로 바꾸는 사람’을 찾습니다.

CORE

문제 정의력

사용자 말과 행동을 의도, 제약, 성공 지표로 번역하는 힘

AI UX

신뢰 설계

근거, 재질문, 오류, 안전 차단, 사람 확인 요청을 설계하는 힘

PORTFOLIO

실험 기록

프로토타입, UT, 로그, 지표, 개선안을 연결해 설명하는 힘

Q2. 빌더, 디자인+엔지니어링 직무는 주류가 될까

정답은 ‘모든 사람이 다 하는 직무’가 아니라, 경계에서 빠르게 검증하는 역할이 커진다는 것입니다.

**앞으로 강해지는 사람은 한 직무를 버린 사람이 아니라, 자기 전문성을 중심에 두고 옆
직무의 산출물을 읽고 검증할 수 있는 사람입니다.**

디자이너는 코드와 데이터의 제약을, 개발자는 사용자 상태와 신뢰를, PM은 시스템과 운영의 현실을 이해해야
합니다.

Q3. AI로 변화하는 직군과 업무 형태

반복 업무는 줄고, 기준을 세우고 검증하고 운영하는 일이 늘어납니다.

Before	문서 작성	→	After	정책·평가 기준 작성
Before	화면 제작	→	After	AI 상태·신뢰 흐름 설계
Before	코드 구현	→	After	AI 생성 코드 검증·테스트
Before	데이터 추출	→	After	데이터 계약·품질 운영
Before	배포 후 대응	→	After	관측성· Runbook ·가드레일

Q4. 패션 플랫폼 취업을 준비한다면

특정 회사의 정답은 없지만, 플랫폼 기업이 공통적으로 확인하는 역량은 분명합니다.

DOMAIN

고객·상품 감각

검색, 추천, 리뷰, 재고, 브랜드, 가격의 맥락을 이해

DATA

지표와 실험

CTR/CVR만이 아니라 저장률, 선택 확신도, 재질문율, 품질 지표를 해석

AI NATIVE

도구+검증+운영

AI 도구 활용, 코드/프롬프트 리뷰, 평가셋, 로그, 보안 감각

면접에서 강한 답: ‘제가 이런 **AI** 기능을 만들었습니다’ 보다 ‘이 문제에서 어떤 기준으로 **AI**를 쓰고, 어떻게 검증했습니다’ 입니다.

10-15 MINUTES

Live Q&A

남은 시간은 여러분의 질문으로 채우겠습니다.

- 지금 지원을 고민하는 이유
- 내 전공/경험과 AI 직무를 연결하는 방법
- 프로젝트 주제, 포트폴리오, 면접 준비
- PM·디자인·개발·데이터·보안 중 어디서 시작할지

마지막으로 남기고 싶은 세 문장

1. AI는 사람을 대체하기보다, 사람이 책임질 문제의 층위를 바꿉니다.
2. AI Native 인재는 도구를 많이 아는 사람이 아니라, 문제-데이터-제품-운영을 연결하는 사람입니다.
3. 지금 가장 좋은 준비는 하나의 문제를 끝까지 정의하고, 만들고, 검증하고, 설명해보는 것입니다.

AI로 문제를 해결한다는 것은, 더 빨리 답하는 것이 아니라
더 책임 있게 배울 수 있는 시스템을 만드는 일입니다.

Sources & Reference Links

자료 제작에 사용한 로컬 파일과 공식/홈페이지 레퍼런스입니다.

- Local: DESIGN-apple.md
- Local: AI 캠퍼스 커리큘럼_통합본.xlsx
- Local: designs/design-tokens.json
- getdesign.md: <https://getdesign.md/>
- AI Campus Prism: <https://aicampus.goorm.io/>
- OpenAI Agents: <https://platform.openai.com/docs/guides/agents>
- MCP: <https://modelcontextprotocol.io/docs/getting-started/intro>
- LangGraph: <https://docs.langchain.com/oss/python/langgraph/overview>
- LlamaIndex: <https://developers.llamaindex.ai/python/framework/>
- Figma AI: <https://www.figma.com/ai/>
- Kubernetes: <https://kubernetes.io/>
- NVIDIA NeMo Guardrails: <https://github.com/NVIDIA-NeMo/Guardrails>
- Claude Code: <https://code.claude.com/docs/en/overview>
- Vercel: <https://vercel.com/>